

Bellman Ford

The Bellman-Ford algorithm is a graph algorithm used to find the shortest paths from a single source **vertex** to all other vertices in a weighted graph, even if the graph contains negative weight edges. Unlike Dijkstra's algorithm, Bellman-Ford can handle negative edge weights and can also detect negative weight cycles in the graph, which makes it very useful in scenarios where such cycles might exist. The algorithm works by repeatedly relaxing all edges in the graph for **V-1 iterations**, where V is the number of vertices, updating the shortest distance to each vertex if a shorter path is found. After these iterations, it performs one additional check on all edges to detect if any distance can still be reduced; if so, a negative weight cycle exists. The time complexity of the Bellman-Ford algorithm is $O(V \times E)$, where V is the number of vertices and E is the number of edges, and its space complexity is $O(V)$. Bellman-Ford is widely used in networking protocols, such as the Distance Vector Routing Algorithm, where shortest path calculation is critical, and in situations where graphs may contain negative edge weights that need to be accounted for safely.

Advantages / Benefits

- Can handle negative edge weights, unlike Dijkstra's algorithm.
- Guarantees shortest path from a single source to all vertices in a weighted graph.
- Can detect negative weight cycles, ensuring correctness.
- Simple to implement and widely used in network routing protocols (e.g., Distance Vector Routing).
- Works for both directed and undirected graphs, making it versatile.

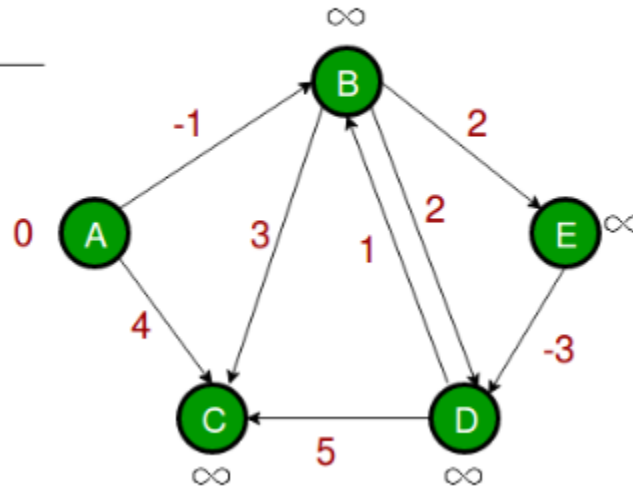
Limitations

- **Time complexity is $O(V \times E)$** , slower than Dijkstra for graphs without negative edges.
- Less efficient for **large or dense graphs** due to repeated edge relaxations.
- Cannot provide shortest paths if a **negative weight cycle exists**.
- Requires **more iterations and checks** compared to some other shortest path algorithms.
- Not suitable for applications needing **very fast shortest path computation** on simple weighted graphs.

Steps:

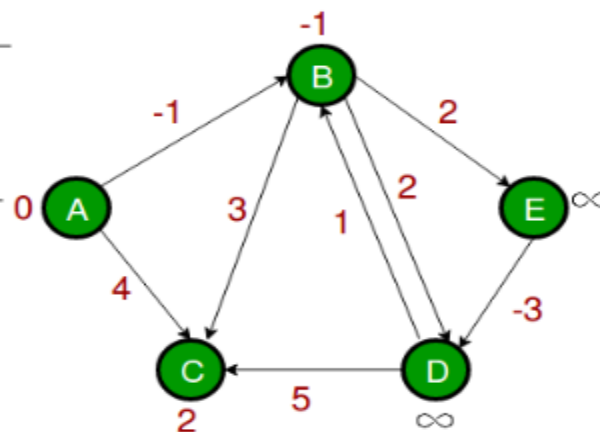
Let the given source vertex be 0. Initialize all distances as infinite, except the distance to source itself. Total number of vertices in the graph is 5, so all edges must be processed 4 times.

A	B	C	D	E
0	∞	∞	∞	∞



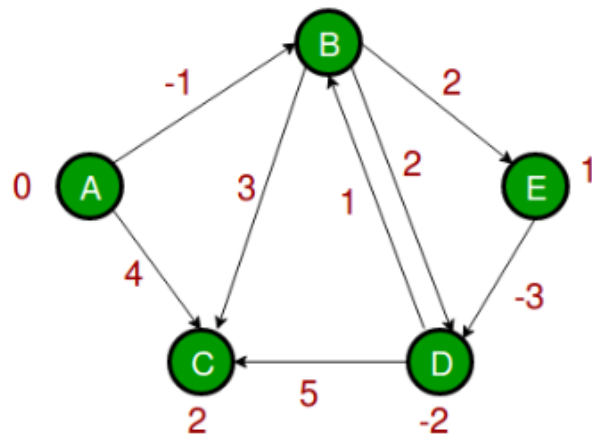
Let all edges are processed in following order: (B, E), (D, B), (B, D), (A, B), (A, C), (D, C), (B, C), (E, D). We get following distances when all edges are processed first time. The first row in shows initial distances. The second row shows distances when edges (B, E), (D, B), (B, D) and (A, B) are processed. The third row shows distances when (A, C) is processed. The fourth row shows when (D, C), (B, C) and (E, D) are processed.

A	B	C	D	E
0	∞	∞	∞	∞
0	-1	∞	∞	∞
0	-1	4	∞	∞
0	-1	2	∞	∞



The first iteration guarantees to give all shortest paths which are at most 1 edge long. We get following distances when all edges are processed second time (The last row shows final values).

	A	B	C	D	E
A	0	∞	∞	∞	∞
B	-1	0	∞	∞	∞
C	-1	4	0	∞	∞
D	-1	2	∞	0	∞
E	-3	1	-2	-2	0



The second iteration guarantees to give all shortest paths which are at most 2 edges long. The algorithm processes all edges 2 more times. The distances are minimized after the second iteration, so third and fourth iterations don't update the distances.

Pseudocode:

BELLMAN_FORD(graph, cost, n, source):

 DECLARE dis[1..n], parent[1..n]

 FOR i = 1 TO n:

 dis[i] = ∞

 parent[i] = -1

 dis[source] = 0

 // Relax edges n-1 times

 FOR k = 1 TO n-1:

 FOR u = 1 TO n:

 FOR EACH neighbor v of u:

 w = weight(u, v)

 IF dis[u] + w < dis[v]:

 dis[v] = dis[u] + w

 parent[v] = u

 PRINT "dis"

 FOR i = 1 TO n:

 PRINT dis[i]

Time Complexity

$$\text{Time Complexity} = O(V \times E)$$

- **V** = number of vertices
- **E** = number of edges

Breakdown:

Operation	Description	Time Complexity
Initialization	Set distance[] and predecessor[] arrays for all vertices	$O(V)$
Edge Relaxation	For each of the $V-1$ iterations, relax all E edges	$O(V \times E)$
Negative Cycle Check	After $V-1$ iterations, check all edges once more to detect negative cycles	$O(E)$
Total Time Complexity	Sum of all above operations	$O(V \times E)$

Bellman Ford Code:

<https://github.com/AHANIK33/Algorithm/blob/main/Bellman%20Ford/Basic/basic.cpp>